VNiVERSiDAD D SALAMANCA

Laboratorio de Mecánica y Ondas | Práctica 8

Redondeo y ajuste por mínimos cuadrados ponderados

Cómo funciona, línea a línea, el código de nuestras gráficas

Daniel Martín Guerrero

Grado en Física | Universidad de Salamanca

octubre de 2026

Qué vamos a ver

1. El redondeo
2. El ajuste por mínimos cuadrados ponderados

> **La idea**
>
> Cuando hago una gráfica con ajuste, el código hace dos cosas que conviene entender bien: **calcula la recta** que mejor describe los datos, teniendo en cuenta lo fiable que es cada punto, y **presenta el resultado** con el número de cifras correcto.

Iré línea a línea, con los números reales del ajuste de la velocidad del sonido (caso 1 del informe), para que cada paso se vea con datos concretos.

PARTE 1

El redondeo

La regla que seguimos

Criterio del informe

- La incertidumbre se da con **una** cifra significativa.
- Si esa cifra es un **1**, conservamos **dos**.
- El valor se redondea a la **misma posición decimal** que la incertidumbre.

Con una sola cifra, 0,0118 quedaría en 0,01 y perderíamos casi un 20 % de información; por eso, si empieza por 1, guardamos dos.

Sin redondear	1.ª cifra	Resultado
$0{,}353\,17 \pm 0{,}011\,8$	1	$0{,}353 \pm 0{,}012$
$244{,}66 \pm 88{,}6$	8	$(2{,}4 \pm 0{,}9) \times 10^2$
$40\,607{,}67 \pm 0{,}357$	3	$40\,607{,}7 \pm 0{,}4$

Los dos primeros ejemplos, A ($u = 0{,}0118$) y B ($u = 88{,}6$), nos acompañarán durante toda esta parte.

La función completa

```python
def redondea(valor, incertidumbre):
    exp = math.floor(math.log10(abs(incertidumbre)))
    primera = int(abs(incertidumbre) / 10**exp)
    decimales = -(exp - 1) if primera == 1 else -exp
    v, u = round(valor, decimales), round(incertidumbre, decimales)
    d = max(decimales, 0)
    return f"{v:.{d}f}".replace(".", ","), f"{u:.{d}f}".replace(".", ",")
```

Recibe el valor y su incertidumbre **sin redondear** y devuelve dos textos listos para escribir en el informe.

Los pasos son: localizar la primera cifra, ver si es un 1, decidir dónde cortar, redondear y escribir con coma decimal.

Paso 1: ¿dónde está la primera cifra?

```
exp = math.floor(math.log10(abs(incertidumbre)))
```

El **logaritmo decimal** responde a una pregunta muy concreta: *¿a qué número tengo que elevar 10 para obtener este?*

Número	Potencia	$\log_{10}$
100	10^2	2
10	10^1	1
1	10^0	0
0,1	10^{-1}	-1
0,01	10^{-2}	-2

Si el número cae entre dos potencias, su logaritmo cae entre los dos enteros:

- 88,6 está entre 10 y 100: $\log_{10}(88{,}6) \approx 1{,}95$.
- 0,0118 está entre 0,01 y 0,1: $\log_{10}(0{,}0118) \approx -1{,}93$.

`abs(...)` solo garantiza que el número sea positivo, porque el logaritmo de un negativo no existe.

Paso 1: el papel de `floor`

`math.floor` redondea **siempre hacia abajo** y se queda con la parte entera, que indica la **posición de la primera cifra significativa**:

	$\log_{10}$	`floor`	Posición
A: 0,0118	$-1{,}93$	-2	centésimas (10^{-2})
B: 88,6	1,95	1	decenas (10^{1})

> **Ojo con los negativos**
>
> «Hacia abajo» significa *alejándose del cero*: $-1{,}93$ pasa a -2, no a -1. Y así es como debe ser, porque el 1 de 0,0118 está en las centésimas, no en las décimas.

A ojo es fácil ver dónde empieza un número; el ordenador no lo «ve», y el logaritmo es la manera matemática de averiguarlo para cualquier valor.

Paso 2: ¿cuál es esa cifra?

```
primera = int(abs(incertidumbre) / 10**exp)
```

Dividir entre 10^{exp} «corre la coma» hasta dejar la primera cifra delante, e `int(...)` corta todo lo que hay detrás:

$$\text{A: } \frac{0{,}0118}{10^{-2}} = 1{,}18 \longrightarrow \mathbf{1}, \qquad \text{B: } \frac{88{,}6}{10^{1}} = 8{,}86 \longrightarrow \mathbf{8}.$$

> Ya sabemos dónde está la primera cifra (`exp`) y cuál es (`primera`). Con eso basta para aplicar la regla.

Paso 3: ¿dónde cortamos?

```
decimales = -(exp - 1) if primera == 1 else -exp
```

Aquí vive la regla:

- Si la primera cifra es un 1, guardamos **dos** cifras: cortamos una posición más a la derecha, `-(exp - 1)`.
- Si no, guardamos **una**: `-exp`.

Un número de decimales negativo significa redondear a la izquierda de la coma: -1 es «a las decenas».

	`exp`	`primera`	`decimales`
A	-2	1	$-(-2-1)=3$
B	1	8	-1

A: tres decimales, hasta las milésimas.
B: hasta las decenas.

Paso 4: redondear y escribir

```
v, u = round(valor, decimales), round(incertidumbre, decimales)
d = max(decimales, 0)
return f"{v:.{d}f}".replace(".", ","), f"{u:.{d}f}".replace(".", ",")
```

- `round` redondea valor e incertidumbre **a la misma posición**, para que sus últimas cifras coincidan.
- `max(decimales, 0)`: si redondeamos a las decenas no hay decimales que escribir, así que usamos 0.
- `f"{v:.{d}f}"` escribe exactamente `d` decimales (conserva ceros como el de 0,0120) y `replace` pone la coma.

	Entrada	Devuelve
A	0,353 17, 0,011 8	`"0,353", "0,012"`
B	244,66, 88,6	`"240", "90"`

El caso B lo escribimos en el informe como $(2{,}4 \pm 0{,}9) \times 10^2$ m/s.

La función `coma` y dos detalles finos

```python
def coma(x, d):
    return f"{x:.{d}f}".replace(".", ",")
```

Solo da formato: escribe `x` con `d` decimales y coma decimal, sin aplicar ninguna regla. La uso para números auxiliares como $t_{95,\nu}$ o χ^2/ν: `coma(2.5706, 2)` devuelve `"2,57"`.

Redondeo de los empates

Python lleva los 5 exactos a la cifra *par*: `round(0.125, 2)` da 0,12. Coincide con la regla de las diapositivas («si es 5, según sea par o impar»).

Un caso límite

$u = 0{,}0196$ empieza por 1 y conserva dos cifras: 0,020. Tras redondear empieza por 2, pero mantiene las dos. Es raro; si aparece, se ajusta a mano.

PARTE 2

El ajuste por mínimos cuadrados ponderados

¿Por qué ponderar?

En el caso 1 ajustamos v frente a $x = 1 - f_E/f_R$. Cada punto es la media de seis lecturas, con $u^2 = s^2/n + \Delta^2/12$, y la x hereda la incertidumbre de las frecuencias. Comparemos las dos:

> La incertidumbre horizontal, llevada a la vertical, es **entre 10 y 150 veces mayor** que la vertical. El ajuste no puede ignorar la x, ni tratar igual todos los puntos.

Serie	$u(y)$ (m/s)	$a\,u(x)$ (m/s
Máxima, acercamiento	0,0026	0,030
Máxima, alejamiento	0,0012	**0,184**
Media, acercamiento	0,0009	0,011
Media, alejamiento	0,0008	0,015
Mínima, acercamiento	0,0004	0,025
Mínima, alejamiento	0,0007	0,008

La función completa (I): la iteración

```
def ajuste_ponderado(x, ux, y, uy, modelo="libre"):
    x, ux, y, uy = (np.asarray(t, float) for t in (x, ux, y, uy))
    n = len(x)
    a = np.polyfit(x, y, 1)[0] if modelo == "libre" else (x @ y) / (x @ x)
    for _ in range(500):
        w = 1 / (uy**2 + a**2 * ux**2)
        if modelo == "libre":
            W = w.sum()
            m = lambda z: (w * z).sum() / W
            a_nueva = (m(x * y) - m(x) * m(y)) / (m(x * x) - m(x) ** 2)
        else:
            a_nueva = (w * x * y).sum() / (w * x * x).sum()
        if abs(a_nueva - a) <= 1e-12 * max(1.0, abs(a)):
            a = a_nueva
            break
        a = a_nueva
```

La función completa (II): resultados

```
ue = np.sqrt(uy**2 + a**2 * ux**2)
w = 1 / ue**2
if modelo == "libre":
    W = w.sum()
    m = lambda z: (w * z).sum() / W
    b = m(y) - a * m(x)
    ua = 1 / math.sqrt(W) / math.sqrt(m(x * x) - m(x) ** 2)
    ub = ua * math.sqrt(m(x * x))
    nu = n - 2
else:
    b, ub, nu = 0.0, 0.0, n - 1
    ua = 1 / math.sqrt((w * x * x).sum())
chi2 = float((w * (y - a * x - b) ** 2).sum())
factor = max(1.0, math.sqrt(chi2 / nu)) if nu > 0 else 1.0
t95 = float(stats.t.ppf(0.975, nu)) if nu > 0 else float("nan")
```

Preparación y pendiente de partida

```
x, ux, y, uy = (np.asarray(t, float) for t in (x, ux, y, uy))
n = len(x)
a = np.polyfit(x, y, 1)[0] if modelo == "libre" else (x @ y) / (x @ x)
```

- `np.asarray` convierte las listas en *arrays*, con los que se opera punto a punto: `x * y` multiplica cada x_i por su y_i.
- `n = len(x)` cuenta los puntos: $n = 6$.

La iteración necesita una pendiente inicial:

- **libre**: el ajuste sin pesos, `np.polyfit`, que da $a_0 = 82{,}4\,\mathrm{m/s}$;
- **origen**: $a = \sum x_i y_i / \sum x_i^2$.

> Ese 82,4 m/s es el ajuste básico del informe, arrastrado por el punto anómalo.

Los pesos: la varianza efectiva

```
w = 1 / (uy**2 + a**2 * ux**2)
```

Peso de cada punto

$$w_i = \frac{1}{u_i^2}, \qquad u_i^2 = u^2(y_i) + a^2 u^2(x_i)$$

Un punto con el doble de incertidumbre pesa **cuatro veces menos**: los precisos mandan en la recta.

¿Por qué a^2? Si muevo un punto δx en horizontal, sobre una recta de pendiente a eso equivale a moverlo $a\,\delta x$ en vertical. Sumando ambas contribuciones como varianzas independientes sale la expresión de la izquierda.

> El problema: para calcular w_i necesito a, que es justo lo que busco. Por eso hay que iterar.

La media ponderada: ¿qué es z?

```
W = w.sum()
m = lambda z: (w * z).sum() / W
```

`lambda` define una función en una línea; z es solo el nombre de su argumento, un **comodín**. `m` es una máquina de medias ponderadas:

$$\langle z \rangle = \frac{\sum_i w_i z_i}{\sum_i w_i}$$

Ejemplo: pesos $(1, 1, 2)$ y valores $(10, 20, 40)$ dan $\langle z \rangle = 110/4 = 27{,}5$, más que la media normal (23,3), porque el 40 pesa el doble.

Llamada	Calcula	Caso 1
`m(x)`	$\langle x \rangle$	$3{,}27 \times 10^{-4}$
`m(y)`	$\langle y \rangle$	$-0{,}0214\,\mathrm{m/s}$
`m(x * x)`	$\langle x^2 \rangle$	$6{,}49 \times 10^{-7}$
`m(x * y)`	$\langle xy \rangle$	$1{,}26 \times 10^{-4}$

La pendiente ponderada

```
a_nueva = (m(x * y) - m(x) * m(y)) / (m(x * x) - m(x) ** 2)
```

Mínimos cuadrados ponderados

$$a = \frac{\langle xy \rangle - \langle x \rangle \langle y \rangle}{\langle x^2 \rangle - \langle x \rangle^2}, \qquad b = \langle y \rangle - a \langle x \rangle$$

Sale de minimizar $\chi^2 = \sum_i w_i \, (y_i - a x_i - b)^2$ respecto de a y b. Arriba hay una «covarianza» entre x e y; abajo, una «varianza» de x, ambas ponderadas.

Para la recta por el origen la idea es la misma:

```
a_nueva = (w*x*y).sum() / (w*x*x).sum()
```

Sin pesos (w_i iguales) recuperamos las fórmulas básicas de las notas: es el mismo ajuste de siempre, solo que cada punto cuenta según su fiabilidad.

¿Cuándo paramos de iterar?

```
if abs(a_nueva - a) <= 1e-12 * max(1.0, abs(a)):
    a = a_nueva
    break
a = a_nueva
```

Si la pendiente nueva apenas difiere de la anterior (una billonésima relativa), ha convergido y salimos con `break`. Si no, la guardamos y damos otra vuelta. El `range(500)` es solo un tope de seguridad.

> El salto de 82 a 244 m/s en la primera vuelta es el efecto de los pesos: el punto anómalo deja de arrastrar la recta.

Vuelta	a (m/s)
Inicio (sin pesos)	82,4
1	244,279
2	244,657
3	244,657 (convergido)

Los pesos finales: quién manda en la recta

```
ue = np.sqrt(uy**2 + a**2 * ux**2)
w = 1 / ue**2
```

Serie	u_i (m/s)	Peso
Mínima, alejamiento	0,008	**51 %**
Media, acercamiento	0,011	26 %
Media, alejamiento	0,015	14 %
Mínima, acercamiento	0,025	5 %
Máxima, acercamiento	0,030	3,5 %
Máxima, alejamiento	0,184	**0,1 %**

Con la pendiente ya convergida recalculo la incertidumbre efectiva de cada punto y su peso definitivo.

> El punto anómalo prácticamente no cuenta, y las series a velocidad media y mínima, las más precisas, deciden la recta.

La ordenada y las incertidumbres de los parámetros

```
b = m(y) - a * m(x)
ua = 1 / math.sqrt(W) / math.sqrt(m(x * x) - m(x) ** 2)
ub = ua * math.sqrt(m(x * x))
nu = n - 2
```

Notas de incertidumbres, ec. (64)

$$u(a) = \frac{1}{\sqrt{\sum_i w_i}\,\sqrt{\langle x^2\rangle - \langle x\rangle^2}}$$

$$u(b) = u(a)\sqrt{\langle x^2\rangle}$$

- $b = \langle y\rangle - a\langle x\rangle$: la recta pasa por el «centro de gravedad» ponderado. Da $b = -0{,}1014\,\mathrm{m/s}$.
- $u(a) = 7{,}67\,\mathrm{m/s}$ y $u(b) = 0{,}0062\,\mathrm{m/s}$: más peso total y más separación en x fijan mejor la pendiente.
- $\nu = 6 - 2 = 4$, porque estimamos dos parámetros.

La bondad del ajuste: χ^2

```python
chi2 = float((w * (y - a * x - b) ** 2).sum())
factor = max(1.0, math.sqrt(chi2 / nu)) if nu > 0 else 1.0
```

Chi-cuadrado

$$\chi^2 = \sum_i \left(\frac{y_i - a x_i - b}{u_i} \right)^2$$

Mide cuántas incertidumbres se separa cada punto de la recta, al cuadrado y sumado. Si las barras están bien estimadas, $\chi^2 \approx \nu$.

En el caso 1, $\chi^2 = 69{,}3$ frente a $\nu = 4$: unas 17 veces más. Los puntos se dispersan más de lo que dicen sus barras, así que $u(a) = 7{,}67\,\mathrm{m/s}$ es demasiado optimista.

> Corregimos con $\sqrt{69{,}3/4} = 4{,}16$. El `max(1.0, ...)` hace que la incertidumbre solo se infle, nunca se encoja.

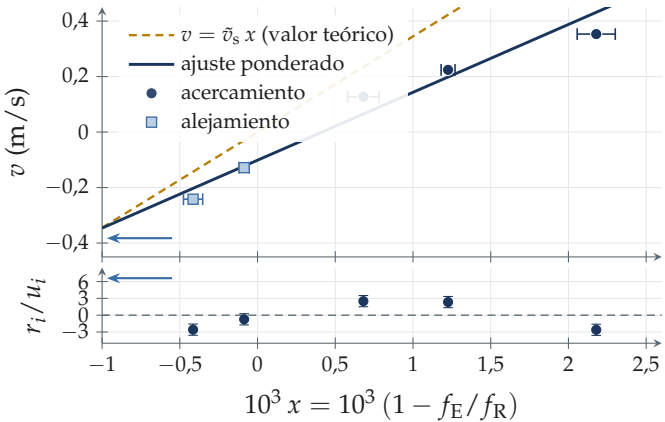
La t de Student y lo que devuelve la función

```
t95 = float(stats.t.ppf(0.975, nu)) if nu > 0 else float("nan")
return dict(a=a, b=b, ua=ua * factor, ub=ub * factor,
            Ua=t95 * ua * factor, Ub=t95 * ub * factor, ...)
```

`stats.t.ppf(0.975, 4)` da el valor que deja un 2,5 % en cada cola: un intervalo central del 95 %. Con solo 4 grados de libertad, $t_{95,4} = 2{,}78$, bastante mayor que el 1,96 de la gaussiana, porque con pocos puntos la propia incertidumbre está mal conocida.

Clave	Contenido	Caso 1
`a`	pendiente	244,657 m/s
`ua`	$u(a)$ inflada	$7{,}67 \times 4{,}16 = 31{,}9$ m/s
`Ua`	$t_{95,4}\, u(a)$	$2{,}78 \times 31{,}9 = 88{,}6$ m/s
`b`, `Ub`	ordenada	$-0{,}101 \pm 0{,}071$ m/s

Todo junto: del código al resultado



Ajuste 1 — emisor móvil

$$v_\mathrm{s} = (2{,}4 \pm 0{,}9) \times 10^2\,\mathrm{m/s}$$

$$b = (-0{,}10 \pm 0{,}07)\,\mathrm{m/s}$$

Al 95 %, con $\nu = 4$ y $t_{95{,}4} = 2{,}78$.

Los números salen de `ajuste_ponderado` y su presentación, de `redondea`: $244{,}657 \pm 88{,}6$ se convierte en $(2{,}4 \pm 0{,}9) \times 10^2$.

En resumen

El redondeo

1. El logaritmo y `floor` localizan la primera cifra.
2. Dividir y truncar nos dice cuál es.
3. Si es un 1, guardamos dos cifras; si no, una.
4. Valor e incertidumbre se cortan en la misma posición y se escriben con coma decimal.

El ajuste ponderado

1. Cada punto pesa $1/u_i^2$, con la varianza efectiva.
2. Se itera hasta que la pendiente no cambia.
3. Se calculan b, $u(a)$ y $u(b)$ con medias ponderadas.
4. χ^2/ν nos dice si inflar las incertidumbres.
5. La t de Student da el intervalo al 95 %.

¡Gracias!

Nada de esto es magia: son las fórmulas de las notas de incertidumbres, escritas para que el ordenador haga las cuentas por nosotros.

Daniel Martín Guerrero

Laboratorio de Mecánica y Ondas | Grado en Física | Universidad de Salamanca